Demo Script — Ultron (5-7 minute walkthrough)

0. Setup (before judges arrive)

- [ ] llama-server running with the Q6_K GGUF model, confirmed responding
- [ ] A test engagement pre-authorized in a staging environment (safe lab target, e.g. an intentionally vulnerable local VM)
- [ ] Terminal/UI visible and font size large enough to read from a few feet away

1. Hook (30s)

"Ultron is a pentesting copilot built on a model we fine-tuned ourselves — Qwen3.5-2B — so it can turn plain-English instructions into the exact tool calls a red-team workflow needs, without a human writing scan commands by hand."

2. Show the model choice, briefly (30-45s)

Pull up the reasoning-index and BFCL charts from the fine-tuning report. "We benchmarked it against Qwen3, Llama-3.2, Gemma-3, and specialist function-calling models before picking Qwen3.5-2B — best reasoning per parameter at this size, native tool calling, and small enough to run fully offline during an engagement."

3. Live demo (2-3 min)

1. **Start an engagement** (show the authorization step first — this is a safety/judging point, don't skip it):

   > "Start an engagement against `lab-target.local`, authorization ref ENG-2026-014."

   - Point out: no scan can run until this step succeeds.

2. **Natural-language recon request**:

   > "Run a port scan on lab-target.local."

   - Show the model's structured tool call (`run_recon_scan`) in the console/log — this is the "look, it's really calling a function, not just chatting" moment.

3. **Trigger the human-confirmation gate** (important safety beat):

   > Ask for something classified higher-risk, e.g. "Try the login form on the discovered admin panel."

   - Show the agent stopping and calling `request_human_confirmation` instead of acting — narrate: "this is the guardrail — nothing above read-only recon executes without a human approving it."

4. **Show a finding + report**:

   > "Log this open port as a finding and generate the report."

   - Show the generated report artifact.

4. Training results, briefly (30-45s)

Pull up the loss/gradient-norm/LR chart. "After fine-tuning on our custom tool-call dataset, training loss dropped from 2.7 to 1.2 over just 30 steps — QLoRA is sample-efficient enough that we didn't need a huge dataset to teach it our tool syntax."

5. Honesty beat (don't skip — judges reward this) (20-30s)

"Two things we're upfront about: we haven't yet run a held-out evaluation, so today's numbers are training-fit, not generalization — that's our next step. And every action in this demo is running inside an authorized lab scope; the orchestration layer, not the model, enforces that boundary."

6. Close (15-20s)

"That's Ultron — a fine-tuned, tool-calling pentest copilot with authorization and human approval built into the loop from day one, not bolted on after."

Anticipated Judge Questions (have answers ready)

Question	Answer pointer
"How do you stop it going out of scope?"	`RESPONSIBLE_USE_POLICY.md` §1 — orchestration layer enforces scope, not the model
"What's your actual accuracy, not just loss?"	`EVALUATION_REPORT.md` §3 — plan is defined, results pending
"Why not a bigger model?"	Fine-tuning report §4 — context length, license, and reasoning/size trade-off
"What if the model hallucinates a tool call?"	`API_INTEGRATION_GUIDE.md` §4 — orchestration layer validates every call before execution
"Is any of this real exploit data?"	`DATASET_CARD.md` — no real exploit payloads or client data in the training set